Universidade Federal do Maranhão

Curso de Engenharia da Computação

PLANEJAMENTO DO PROJETO

Sistema Distribuído para Auxílio de Dados Hospitalares

Relatório de Implementação e Atualização do Cronograma: Marcos 1 e 2

Curso: Engenharia da Computação

Disciplina: Sistemas Distribuídos

Professor(a): Luiz Henrique Neves Rodrigues

Grupo: Fernando da Silva Costa; Samara Santos Viegas; Eduardo dos Santos Oliveira; Herick Vinicius Pinheiro Da Conceição; Vyktor Gabryel Costa de Souza

28 de setembro de 2026

Sumário

1 Introdução e Visão Geral **2**

2 Diagrama de Gantt **2**

3 Análise e Modelagem do Dataset Hospitalar **3**
3.1 De Onde Vêm os Dados . . . 3
3.2 Traduzindo o Dataset para a Nossa API (Schema Mapping) . . . 3

4 Marco 1 — Simulação e Servidor Base **4**
4.1 Como a Informação Percorre o Sistema . . . 4
4.2 O Produtor de Telemetria (`client/csv_sender.py`) . . . 4
4.3 O Servidor Base (`server/app.py`) . . . 5
4.4 Onde os Dados Ficam Guardados (`server/database.py`) . . . 5

5 Marco 2 — Balanceamento de Carga e Múltiplas Instâncias **5**
5.1 A Arquitetura que Montamos . . . 5
5.2 O Algoritmo Round Robin . . . 6
5.3 Como Cada Servidor se Identifica . . . 6
5.4 Bancos de Dados Independentes . . . 7

6 Decisões de Engenharia e Desafios Superados **7**

7 Validação e Resultados Experimentais **7**

8 Roteiro Prático de Demonstração e Verificação **8**
8.1 Primeiro Passo: Ambiente e Testes Rápidos . . . 8
8.2 Demonstração do Marco 1 (Servidor Base) . . . 8
8.3 Demonstração do Marco 2 (Balanceamento de Carga) . . . 8
8.4 Evidências de Funcionamento . . . 9

9 Conclusão **9**

1 Introdução e Visão Geral

Em um hospital, os pacientes são monitorados o tempo todo, e cada monitor gera um fluxo constante de informações importantes: sinais vitais, alertas e ocorrências de emergência. Se tudo isso chega a um único servidor central, ele pode ficar sobrecarregado, responder mais devagar e, pior, se ele cair, o sistema inteiro para (o chamado *Single Point of Failure*).

Foi pensando nesse problema que nosso grupo construiu o **Sistema Distribuído para Auxílio de Dados Hospitalares**. O projeto tem finalidade acadêmica e didática, e nós o desenvolvemos aos poucos, em marcos evolutivos. Neste documento contamos como foram os dois primeiros:

- **Marco 1 — Simulação e Servidor Base:** lemos, pouco a pouco, a telemetria de um conjunto de dados hospitalar (*dataset* do Kaggle), validamos cada mensagem em uma API RESTful (FastAPI), convertemos os tipos de dados e guardamos tudo em um banco local (SQLite).
- **Marco 2 — Balanceamento de Carga e Múltiplas Instâncias:** colocamos três instâncias do servidor para rodar ao mesmo tempo e criamos um Balanceador de Carga que distribui as requisições pelo algoritmo *Round Robin*, de forma transparente para quem envia os dados e com identificação clara de qual servidor processou cada mensagem.

Todo o projeto foi escrito em Python (versão 3.10 ou superior). Procuramos manter o código claro, com as partes bem separadas e fácil de rodar em qualquer máquina.

2 Diagrama de Gantt

A Figura 1 apresenta o diagrama de Gantt atualizado do projeto. O cronograma vai de 18/08/2026 a 01/12/2026 e está organizado em quatro fases: Planejamento, Execução, Monitoramento e Encerramento. As barras em verde são as atividades concluídas, as em cinza ainda não foram iniciadas, e a linha tracejada ("Hoje") marca a data desta atualização.

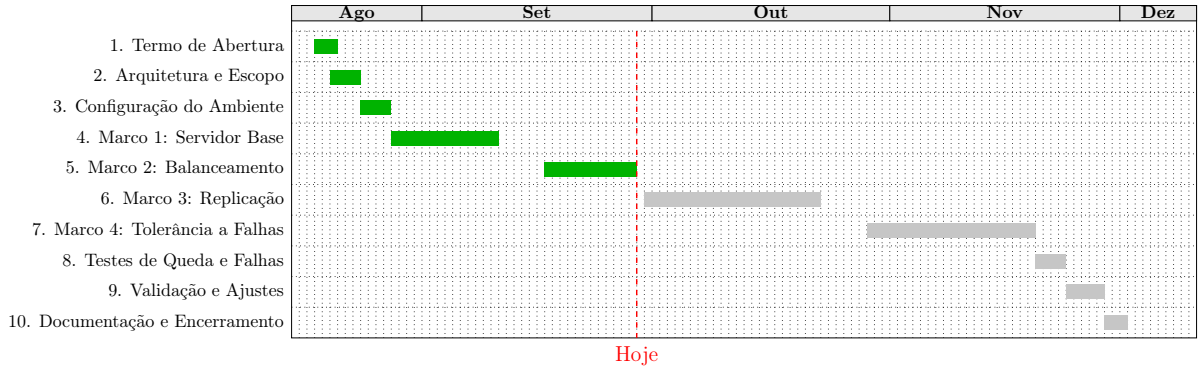


Figura 1: Diagrama de Gantt do projeto, atualizado em 28/09/2026.

Resumindo onde estamos hoje:

- **Planejamento (atividades 1 a 3):** concluído. Já elaboramos o Termo de Abertura, definimos a arquitetura e o escopo e deixamos o ambiente de desenvolvimento configurado.
- **Marco 1 — Simulação e Servidor Base:** concluído.
- **Marco 2 — Balanceamento de Servidores:** concluído no dia 28/09/2026. Já temos as três instâncias do servidor rodando com o balanceador Round Robin distribuindo as requisições entre elas.
- **Marcos 3 e 4 (Replicação de Dados e Tolerância a Falhas):** ainda não foram iniciados. O Marco 3 começa no dia 30/09/2026, e o Marco 4 vem logo em seguida.
- **Monitoramento e Encerramento (atividades 8 a 10):** ainda não iniciados, com testes de queda e falhas, validação final e documentação previstos para novembro.

3 Análise e Modelagem do Dataset Hospitalar

3.1 De Onde Vêm os Dados

Os dados que usamos vêm de um arquivo CSV do Kaggle:

`Synthetic_patient-HealthCare-Monitoring_dataset.csv`

Em vez de gerar números aleatórios durante a execução, preferimos trabalhar com telemetria já coletada, que deixa a simulação mais próxima da realidade.

Ao explorarmos o arquivo, encontramos o seguinte:

- **Volume:** 60.000 registros clínicos (60.001 linhas, contando o cabeçalho).
- **Completude:** nenhum valor ausente em nenhuma coluna (0% de dados nulos).
- **Sem carimbo de tempo:** o arquivo original não tem uma coluna de data e hora. Por isso, o servidor registra o *timestamp* no exato momento em que recebe cada mensagem.

3.2 Traduzindo o Dataset para a Nossa API (Schema Mapping)

Como o arquivo está em inglês e tem caracteres especiais nos nomes das colunas, criamos um mapeamento no módulo `server/models.py` para usar nomes em português, padronizados e mais adequados a uma API RESTful. A Tabela 1 mostra a correspondência.

Tabela 1: Mapeamento de Atributos do CSV Kaggle para a API e Banco de Dados

Coluna Original (CSV)	Campo na API/DB	Tipo	Descrição Semântica
`Patient Number`	`paciente_id`	`int`	Identificador do paciente
`Heart Rate (bpm)`	`frequencia_cardiaca`	`int`	Frequência cardíaca (bpm)
`SpO2 Level (%)`	`spo2`	`int`	Saturação de oxigênio
`Systolic Blood Pressure`	`pressao_sistolica`	`int`	Pressão sistólica (mmHg)
`Diastolic Blood Pressure`	`pressao_diastolica`	`int`	Pressão diastólica (mmHg)
`Body Temperature (°C)`	`temperatura`	`float`	Temperatura corpórea (°C)
`Fall Detection`	`deteccao_queda`	`str`	Indicador de queda (Yes/No)
`Predicted Disease`	`doenca_prevista`	`str`	Diagnóstico clínico previsto
`Data Accuracy (%)`	`precisao_dados`	`int`	Confiabilidade do sensor (%)
`Heart Rate Alert`	`alerta_freq_cardiaca`	`str`	Alerta de FC (NORMAL/ABNORMAL)
`SpO2 Level Alert`	`alerta_spo2`	`str`	Alerta de SpO2
`Blood Pressure Alert`	`alerta_pressao`	`str`	Alerta de pressão arterial
`Temperature Alert`	`alerta_temperatura`	`str`	Alerta de temperatura

4 Marco 1 — Simulação e Servidor Base

4.1 Como a Informação Percorre o Sistema

No Marco 1 montamos o caminho mais simples possível para a telemetria: o dado sai do arquivo, chega ao servidor e é gravado no banco. A Figura 2 mostra esse trajeto.



Figura 2: Fluxo de telemetria do Marco 1: do arquivo tabular à persistência local.

4.2 O Produtor de Telemetria (`client/csv_sender.py`)

Esse script faz o papel de um monitor biomédico à beira do leito, enviando os dados dos pacientes. Ele faz o seguinte:

1. abre o arquivo CSV de forma segura, com `csv.DictReader`;
2. converte cada linha com a função `converter_registro_csv`, transformando os textos em números inteiros ou decimais;
3. envia cada registro ao servidor por uma requisição HTTP `POST`, com o corpo em JSON;
4. espera um intervalo configurável entre os envios (`time.sleep(intervalo)`), o que permite simular tanto em tempo real quanto de forma acelerada;
5. trata falhas de conexão (`ConnectionError` e `Timeout`), para que um problema momentâneo na rede não derrube todo o processo.

4.3 O Servidor Base (`server/app.py`)

O servidor foi feito com o **FastAPI**, um framework assíncrono, e oferece três rotas principais:

- `POST /dados`: recebe os dados do paciente já validados pelo modelo `DadosPaciente` (Pydantic). O servidor adiciona o `timestamp` da máquina e o seu identificador (`server_id`), grava no banco e devolve HTTP 200 com o registro salvo.
- `GET /dados`: devolve os registros armazenados, com paginação pelos parâmetros `limit` e `offset`, o que evita sobrecarregar a memória quando há dezenas de milhares de linhas.
- `GET /dados/{paciente_id}`: devolve o histórico de um paciente específico, ou HTTP 404 se ainda não houver medições dele.

4.4 Onde os Dados Ficam Guardados (`server/database.py`)

Para a persistência usamos o **SQLite3**. Cada instância do servidor cuida do seu próprio arquivo de banco e cria a tabela `dados_pacientes` ao iniciar:

```
        CREATE TABLE IF NOT EXISTS dados_pacientes (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        paciente_id INTEGER NOT NULL,
        frequencia_cardiaca INTEGER NOT NULL,
        spo2 INTEGER NOT NULL,
        pressao_sistolica INTEGER NOT NULL,
        pressao_diastolica INTEGER NOT NULL,
        temperatura REAL NOT NULL,
        deteccao_queda TEXT NOT NULL,
        doenca_prevista TEXT NOT NULL,
        precisao_dados INTEGER NOT NULL,
        alerta_freq_cardiaca TEXT NOT NULL,
        alerta_spo2 TEXT NOT NULL,
        alerta_pressao TEXT NOT NULL,
        alerta_temperatura TEXT NOT NULL,
        timestamp TEXT NOT NULL,
        server_id TEXT NOT NULL
        );
```

Listing 1: Estrutura da Tabela no SQLite

5 Marco 2 — Balanceamento de Carga e Múltiplas Instâncias

5.1 A Arquitetura que Montamos

No Marco 2 o sistema passou a crescer na horizontal. Colocamos um nó intermediário, o Balanceador de Carga, na porta 8000. Ele fica entre o cliente que envia os dados e os servidores de aplicação, que rodam nas portas 8001, 8002 e 8003.

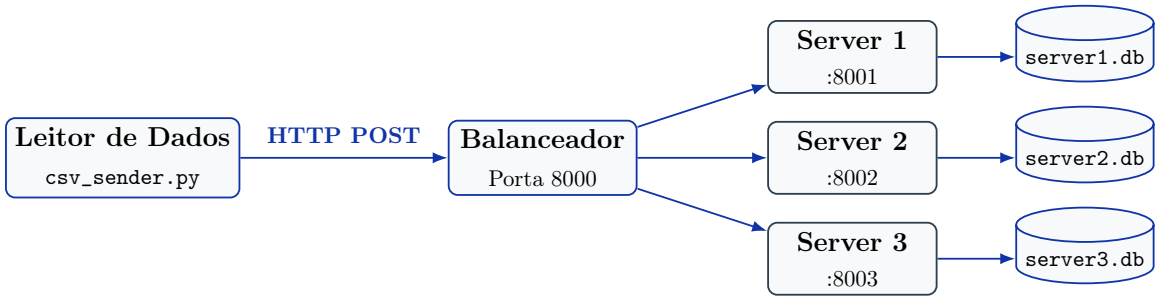


Figura 3: Topologia física e lógica do Marco 2: balanceamento em anel uniforme.

5.2 O Algoritmo Round Robin

O **Round Robin** funciona como um revezamento: as requisições são entregues aos servidores um de cada vez, em ordem, e quando a lista acaba voltamos ao primeiro. Formalmente, dado um conjunto fixo de N servidores $\mathcal{S} = \{s_0, s_1, \ldots, s_{N-1}\}$ e um contador de requisições $r \in \mathbb{N}$ (que começa em 0):

$$\text{Servidor Alvo}(r) = s_r \pmod N \tag{1}$$

Com essa regra, a carga se divide de forma igual entre os servidores:

- Requisição 1 ($r = 0$): Servidor 0 (`server-01`, porta 8001)
- Requisição 2 ($r = 1$): Servidor 1 (`server-02`, porta 8002)
- Requisição 3 ($r = 2$): Servidor 2 (`server-03`, porta 8003)
- Requisição 4 ($r = 3$): Servidor 0 (`server-01`, porta 8001)

Implementamos o balanceador na classe `RoundRobinBalancer`, no módulo `load_balancer/balancer` e usamos a biblioteca `httpx.AsyncClient` para encaminhar as requisições de forma assíncrona.

5.3 Como Cada Servidor se Identifica

As três instâncias rodam exatamente o mesmo código, mas cada uma recebe pela linha de comando a sua própria identidade e o seu próprio banco:

- **Instância 1:** porta `8001`, ID `"server-01"`, banco `databases/server1.db`
- **Instância 2:** porta `8002`, ID `"server-02"`, banco `databases/server2.db`
- **Instância 3:** porta `8003`, ID `"server-03"`, banco `databases/server3.db`

Quando o balanceador repassa a requisição e recebe a resposta do servidor escolhido, ele devolve ao cliente um HTTP 200 que traz o campo:

```
"server_id": "server-01"
```

Assim, o cliente consegue conferir em tempo real como a carga está sendo dividida, sem perder a *transparência de localização*: ele continua falando somente com o endereço `http://localhost:8000`.

5.4 Bancos de Dados Independentes

Como definido no planejamento da disciplina, **ainda não implementamos replicação nem sincronização entre os bancos** nesta etapa.

Cada servidor grava apenas no seu banco SQLite local (`server1.db`, `server2.db` ou `server3.db`). Por isso, se o balanceador enviar 12 registros, cada um dos 3 bancos guardará exatamente 4. A persistência distribuída consistente e a replicação entre os nós ficam para o Marco 3.

6 Decisões de Engenharia e Desafios Superados

Ao longo dos Marcos 1 e 2 tomamos algumas decisões importantes, e algumas delas nasceram de problemas que encontramos pelo caminho:

1. **Troca de um recurso descontinuado do FastAPI:** as versões recentes do FastAPI deixaram de recomendar o decorador `@app.on_event("startup")`. Passamos a usar o gerenciador de ciclo de vida assíncrono (`lifespan`) e chamamos `init_db()` antecipadamente, de forma idempotente. Isso resolveu os erros de tabela inexistente nos testes com `TestClient`.

2. **Respeito ao dataset original:** o CSV de 4,4 MB não foi copiado nem alterado. Criamos um *symbolic link* (`data/dataset.csv`) apontando para o arquivo original, o que mantém tudo reproduzível sem gastar espaço extra.

3. **Testes do Marco 2 sem conflito de portas:** ao testar servidores e balanceadores reais em segundo plano, reaproveitar portas TCP rapidamente causava o erro `Errno 98: Address already in use`. Resolvemos usando *fixtures* de escopo de classe (`scope="class"`) no `pytest`, com portas dinâmicas na faixa 19000. Com isso, sumiram as condições de corrida e os travamentos.

7 Validação e Resultados Experimentais

Para termos segurança de que tudo funciona, escrevemos 26 testes automatizados com `pytest`, divididos em duas suítes.

Tabela 2: Sumário da Suíte de Testes Automatizados

Módulo de Teste	**Qtd.**	**Objetivo da Validação**
`tests/test_marco1.py`	17	Existência do CSV, presença de colunas, parsing sem perda, validação Pydantic, inserção no SQLite, consultas com paginação (`limit/offset`) e busca por paciente.
`tests/test_marco2.py`	9	Aritmética modular do Round Robin, inicialização simultânea de 3 servidores e do balanceador, roteamento HTTP transparente e divisão homogênea entre os bancos.
Total de Testes Aprovados	**26/26**	**Taxa de Sucesso: 100% (tempo de execução: 0,92s)**

8 Roteiro Prático de Demonstração e Verificação

Este roteiro reúne os comandos usados para executar e verificar os Marcos 1 e 2.

8.1 Primeiro Passo: Ambiente e Testes Rápidos

No terminal principal, ative o ambiente virtual e rode a suíte de testes:

```
cd "Projeto SD"
source venv/bin/activate
python -m pytest tests/ -v
```

8.2 Demonstração do Marco 1 (Servidor Base)

Para executar o Marco 1:

1. **Terminal 1 (Servidor):**

```
python server/app.py --port 8001 --server-id server
    -01 --db-path databases/server1.db
```

2. **Terminal 2 (Envio Direto):**

```
python client/csv_sender.py --url http://localhost
    :8001/dados --intervalo 0.3 --limite 5
```

8.3 Demonstração do Marco 2 (Balanceamento de Carga)

O Marco 2 usa cinco terminais: um para cada servidor, um para o balanceador e um para o cliente:

- **Terminal 1 (Server 1):**

```
python server/app.py --port 8001 --server-id server
    -01 --db-path databases/server1.db
```

- **Terminal 2 (Server 2):**

```
python server/app.py --port 8002 --server-id server
    -02 --db-path databases/server2.db
```

- **Terminal 3 (Server 3):**

```
python server/app.py --port 8003 --server-id server
    -03 --db-path databases/server3.db
```

- **Terminal 4 (Balanceador):**

```
python load_balancer/balancer.py
```

- **Terminal 5 (Cliente Produtor):** Envie um lote de 12 requisições pelo balanceador (porta 8000):

```
python client/csv_sender.py --url http://localhost
    :8000/dados --intervalo 0.4 --limite 12
```

8.4 Evidências de Funcionamento

1. **O revezamento no balanceador (Terminal 4):** O balanceador mostra o ciclo das requisições passando de servidor em servidor:

```
[BALANCEADOR] Requisicao 1 -> http://localhost:8001
[BALANCEADOR] Requisicao 2 -> http://localhost:8002
[BALANCEADOR] Requisicao 3 -> http://localhost:8003
[BALANCEADOR] Requisicao 4 -> http://localhost:8001
[BALANCEADOR] Requisicao 5 -> http://localhost:8002
...
```

2. **A confirmação no cliente (Terminal 5):** O leitor informa qual servidor respondeu cada requisição:

```
[    1] Paciente     1 -> server-01 | FC=98  SpO2=96 PA=120/86 Temp
    =38.1
[    2] Paciente     2 -> server-02 | FC=105 SpO2=97 PA=177/104 Temp
    =37.6
[    3] Paciente     3 -> server-03 | FC=90  SpO2=85 PA=139/57 Temp
    =37.0
[    4] Paciente     4 -> server-01 | FC=102 SpO2=87 PA=101/77 Temp
    =36.4
```

3. **A divisão exata nos três bancos SQLite:** Abra um terminal adicional e conte os registros de cada banco:

```
echo "--- Server 1 (8001) ---"
sqlite3 databases/server1.db "SELECT COUNT(*) FROM
    dados_pacientes;"
echo "--- Server 2 (8002) ---"
sqlite3 databases/server2.db "SELECT COUNT(*) FROM
    dados_pacientes;"
echo "--- Server 3 (8003) ---"
sqlite3 databases/server3.db "SELECT COUNT(*) FROM
    dados_pacientes;"
```

Resultado esperado: cada banco terá exatamente **4 registros**, somando os 12 enviados. Isso mostra que a carga foi dividida por igual e que cada instância guarda seus dados de forma isolada.

9 Conclusão

Os objetivos dos Marcos 1 e 2 foram alcançados, e o cronograma atualizado na Seção 2 mostra que o projeto segue dentro do previsto. Com esse trabalho, conseguimos aplicar na prática conceitos importantes de Sistemas Distribuídos:

- heterogeneidade e acoplamento fraco entre os componentes, por meio do protocolo HTTP e do formato JSON;
- uso de dados reais de telemetria, sem precisar recorrer a dados sintéticos desnecessários;
- balanceamento de carga Round Robin com transparência de acesso e tolerância a falhas na camada do cliente;

- uma estrutura modular, que prepara o caminho para o Marco 3 (sincronização e replicação de dados).

Daqui para frente, nosso foco será o Marco 3, começando pela replicação dos dados entre os servidores, e depois a tolerância a falhas do Marco 4.